

Telemetry Weather Station: основной firmware

Документ описывает новую основную прошивку станции на ESP32-S3 и PCB_ESP32S3_V2: где настраивать датчики, как работает адресная система, как формируются флаги, JSON и старый MQTT string payload.

Главные файлы

src/main.cpp - основной код станции. Он создает датчики по конфигурации, опрашивает RS485, синхронизирует время, формирует payload и отправляет MQTT.

config/Configuration_Sensors.h - главный файл для выбора состава станции. Здесь задается количество датчиков каждого типа. 0 означает, что такого датчика на станции нет.

config/Configuration_ModbusAddresses.h - единая карта Modbus-адресов для всех датчиков станции.

config/Configuration_Network.h - Wi-Fi, NTP и MQTT настройки.

config/Configuration_Telemetry.h - период опроса, включение JSON/string payload, батарея.

config/Configuration.h - общий ID станции и базовая системная конфигурация.

config/Configuration_PCB.h и pcb/PCB_ESP32S3_V2.h - пины и аппаратная конфигурация платы. Основной код рассчитан только на PCB_ESP32S3_V2.

Как выбрать датчики станции

В config/Configuration_Sensors.h есть блок Station sensor counts.

Пример:

```
#define RIKA_LEAF_SENSOR_COUNT          1
#define SMALL_LEAF_TEMP_HUMIDITY_COUNT  0
#define RIKA_SOIL3IN1_COUNT             1
#define JXBS_WATER_CONDUCTIVITY_COUNT   0
```

Логика такая:

0 - датчик не установлен, объект драйвера не создается, в telemetry его нет.

1 - создается один датчик этого типа, используется первый адрес из адресной таблицы.

2, 3, ... - создается несколько датчиков этого типа, используются первые N адресов из адресной таблицы.

Если количество больше разрешенного максимума, прошивка не соберется из-за static_assert. Это сделано специально, чтобы ошибка конфигурации была видна до загрузки в контроллер.

Адресная система

Адреса находятся в config/Configuration_ModbusAddresses.h.

Адреса записаны как hex-значения:

0x10..0x19 - почва.

0x20..0x29 - датчики листа.

0x30..0x34 - скорость ветра.

0x35..0x39 - направление ветра.

0x40..0x49 - UV, PAR, солнечная радиация, испарение.

0x50..0x59 - водные датчики.

0x60..0x69 - газовые датчики и газовые shield.

0x70..0x74 - PM / air quality shield.

Важно: 0x50 - это hex адрес 0x50, а не десятичное число 50. Такая схема совпадает с уже проверенными водными датчиками: pH 0x50, EC 0x51, suspended solids 0x52.

Один физический RS485 датчик занимает один Modbus-адрес. Если плата/shield возвращает несколько измерений, она все равно занимает один физический адрес.

Как адреса связываются с количеством датчиков

В Configuration_Sensors.h у каждого типа есть массив адресов.

Пример для почвы:

```
constexpr uint8_t RIKA_SOIL3IN1_ADDRESSES[STATION_MAX_RIKA_SOIL3IN1_SENSORS] = {  
  ADDR_SOIL_00,  
  ADDR_SOIL_01,  
  ADDR_SOIL_02,  
  ADDR_SOIL_03,  
  ADDR_SOIL_04  
};
```

Если RIKA_SOIL3IN1_COUNT равен 3, firmware создаст:

Soil1 на ADDR_SOIL_00.

Soil2 на ADDR_SOIL_01.

Soil3 на ADDR_SOIL_02.

Остальные адреса из массива не используются.

Что происходит в setup()

1. Запускает debug Serial.
2. Инициализирует линии питания датчиков.
3. Инициализирует RS485 интерфейсы.
4. Запускает Modbus bus для RS485_PORT_INDEX_0.

5. Вызывает `buildRuntimeSensors()`.
6. Печатает карту датчиков: ID, короткий ключ, Modbus-адрес, `sample rate`.
7. Подключается к Wi-Fi и синхронизирует UTC-время через NTP.
8. Сразу разрешает первый цикл опроса.

Runtime registry и flag array

В `src/main.cpp` есть структура:

```
struct RuntimeSensor {  
    SensorDriver* driver;  
    SensorKind kind;  
    char id[32];  
    char key[32];  
    bool configured;  
    bool lastReadOk;  
};
```

Это и есть runtime-список датчиков станции.

`configured` означает, что датчик создан по конфигурации.

`lastReadOk` - это флаг последнего цикла. Перед каждым циклом он сбрасывается в `false`. Если датчик ответил и драйвер принял данные как валидные, флаг становится `true`.

Таким образом:

`flags.SensorName = true` - датчик был сконфигурирован и успешно ответил в этом цикле.

`flags.SensorName = false` - датчик был сконфигурирован, но в этом цикле не ответил или данные не прошли проверку.

Данные датчика добавляются в `payload` только если `lastReadOk == true`.

Цикл работы

Основной цикл задается `TELEMETRY_CYCLE_INTERVAL_MS` в `config/Configuration_Telemetry.h`.

По умолчанию:

```
#define TELEMETRY_CYCLE_INTERVAL_MS 600000UL
```

Это 10 минут.

Каждый цикл:

1. Проверяет, не нужно ли обновить время.
2. Включает нужную линию питания датчиков.
3. Ждет `warm-up`.

4. Включает нужный RS485 интерфейс.
5. Последовательно опрашивает все сконфигурированные датчики.
6. Для каждого датчика обновляет lastReadOk.
7. Формирует JSON payload.
8. Формирует legacy string payload, если он включен.
9. Отправляет payload по MQTT.

Время и RTC ESP32-S3

ESP32-S3 использует системное время, которое после синхронизации хранится внутри RTC/системного времени ESP32.

В setup() firmware делает NTP sync через интернет.

После этого firmware обновляет время раз в 24 часа:

```
#define TIME_SYNC_INTERVAL_MS 86400000UL
```

В payload отправляются оба варианта:

timestamp - Unix time UTC.

datetime_utc - читаемый UTC формат YYYY-MM-DD HH:MM:SS.

Wi-Fi и интернет-проверка

Настройки находятся в config/Configuration_Network.h.

```
#define WIFI_SSID    "CHANGE_ME_WIFI"  
#define WIFI_PASSWORD "CHANGE_ME_PASSWORD"
```

Перед NTP или MQTT firmware:

1. Проверяет Wi-Fi подключение.
2. Если Wi-Fi нет, подключается к роутеру/hotspot.
3. Проверяет DNS через WiFi.hostByName().
4. Если интернет выглядит доступным, продолжает NTP или MQTT.

MQTT

Настройки:

```
#define MQTT_HOST      "oxus2.amudar.io"  
#define MQTT_PORT      1883  
#define MQTT_USERNAME  "admin"
```

```
#define MQTT_PASSWORD      "inha2016"
#define MQTT_JSON_TOPIC_PREFIX "stations"
#define MQTT_STRING_TOPIC   "meteodb"
```

JSON payload отправляется в topic:

stations/<STATION_ID>/json

Legacy string payload отправляется в:

meteodb

JSON payload

Пример структуры:

```
{
  "station_id": "station_001",
  "timestamp": 1776672000,
  "datetime_utc": "2026-04-20 10:00:00",
  "flags": {
    "Leaf1": true,
    "Soil1": false,
    "WaterEC1": true
  },
  "data": {
    "Leaf1_temp": 24.10,
    "Leaf1_wet": 0.00,
    "WaterEC1_temp": 25.70,
    "WaterEC1_us_cm": 12.59
  }
}
```

flags показывает статус всех сконфигурированных датчиков.

data содержит только те значения, которые реально прочитались в этом цикле.

Legacy string payload

Старый формат похож на прежний код Arduino/SIM808, но отправка идет через Wi-Fi MQTT.

Пример:

```
meteometric,stationID=station_001
Leaf1=ok,Leaf1_temp=24.10,Leaf1_wet=0.00,Soil1=ok,Soil1_temp=21.30,Soil1_vwc=34.20,Soil1_ec=0.1200
1776672000000000000
```

Последнее число - Unix timestamp с добавленными наносекундами 000000000, как в старом формате.

Короткие ключи telemetry

Leaf1_temp, Leaf1_wet - Rika leaf sensor.

LeafSurface1_temp, LeafSurface1_hum - большой JXBS leaf surface sensor.

SmallLeaf1_temp, SmallLeaf1_hum - маленький датчик температуры/влажности листа.

Soil1_temp, Soil1_vwc, Soil1_ec - Rika soil 3-in-1.

Soil71_temp, Soil71_vwc, Soil71_ec, Soil71_ph, Soil71_n, Soil71_p, Soil71_k - JXBS soil 7-in-1.

WindS1_ms - скорость ветра.

WindD1_deg - направление ветра.

UV1_w_m2 - UV.

PAR1_value - PAR.

Solar1_w_m2 - total solar radiation.

Evap1_value - evaporation.

WaterPH1_temp, WaterPH1_ph - water pH.

WaterEC1_temp, WaterEC1_us_cm - water EC / conductivity.

WaterSS1_temp, WaterSS1_mg_l - suspended solids.

AirQ1_temp, AirQ1_hum, AirQ1_pm25, AirQ1_pm10, AirQ1_tvoc - air quality shield.

GasA1_co, GasA1_o3, GasA1_nh3 - O3/CO/NH3 shield.

GasB1_so2, GasB1_no2, GasB1_pressure_mbar - SO2/NO2/pressure shield.

Battery voltage

Батарея подготовлена, но пока отключена:

```
#define BATTERY_MONITOR_ENABLED false  
#define BATTERY_ADC_PIN -1
```

Когда будет подтвержден ADC pin и делитель на плате, нужно включить BATTERY_MONITOR_ENABLED, указать BATTERY_ADC_PIN и проверить коэффициенты делителя.

Если батарея включена и прочиталась, firmware добавит:

battery_v в JSON root.

Batt в JSON data и legacy string.

Как добавить новый тип датчика

1. Добавить или проверить драйвер в lib/Sensors/....
2. Добавить include в src/main.cpp.
3. Добавить новый SensorKind.

4. Добавить *_COUNT, STATION_MAX_* и адресный массив в Configuration_Sensors.h.
5. Добавить адреса или alias в Configuration_ModbusAddresses.h.
6. Добавить функцию регистрации или расширить существующую регистрацию.
7. Добавить поля в appendSensorJsonData().
8. Собрать station_esp32s3_v2.

Как собрать основной firmware

Среда PlatformIO:

station_esp32s3_v2

Команда:

```
pio run -e station_esp32s3_v2
```

Если запускать из обычного PowerShell:

```
C:\Users\user\.platformio\penv\Scripts\pio.exe run -e station_esp32s3_v2
```

Что уже убрано

Основной firmware больше не рассчитан на Mega2560.

Основной firmware рассчитан на ESP32-S3 и PCB_ESP32S3_V2.

Rain gauge sensors удалены из проекта, потому что такого типа датчиков в станции нет.

Маленький датчик листа в Telemetry Weather Station называется нейтрально: SmallLeafTemperatureHumidity, без Honde в названии.

Что еще нужно решить позже

OTA update пока не реализован. Его лучше добавить отдельным этапом после стабилизации основного MQTT/JSON цикла.

Deep sleep пока не включен. Сейчас firmware работает циклически через loop() и TELEMETRY_CYCLE_INTERVAL_MS.

Battery ADC pin нужно подтвердить по PCB.

Wi-Fi SSID/password нужно заменить перед реальным тестом.